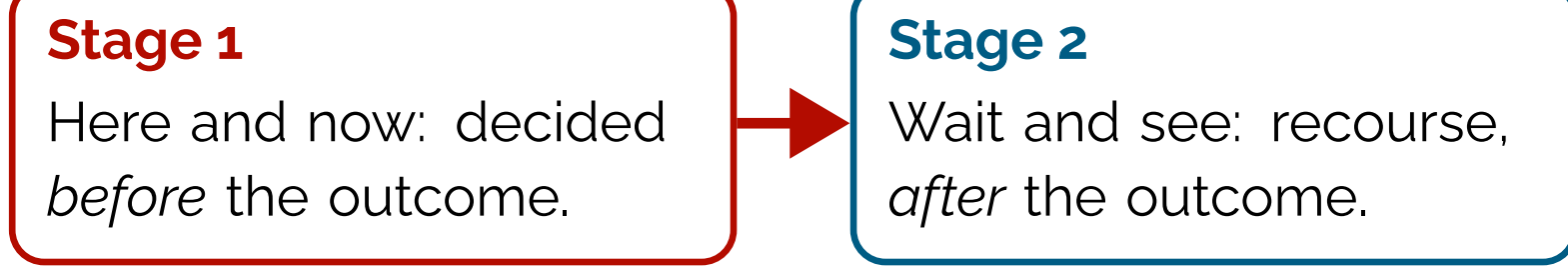


Introduction

Many decisions must be committed to *before* the relevant uncertainty is resolved. A two-stage stochastic program splits the problem into a **here-and-now** decision, then a **wait-and-see** recourse action once a scenario is realised [1].



Stage 1: $\min_x c^\top x + \mathbb{E}_\xi[Q(x, \xi)] \quad \text{s.t. } Ax \geq b, x \geq 0$

Stage 2: $Q(x, \xi) = \min_y q(\xi)^\top y \quad \text{s.t. } W(\xi)y \geq h(\xi) - T(\xi)x, y \geq 0$

With finitely many scenarios this collapses into one optimisation problem, which is the **deterministic equivalent**, but stacks *one full copy* of the recourse problem per scenario into a single model.

Case Study: Pre-Positioning for Disaster Relief

This case study is based on hurricane response logistics [2]. In **Stage 1**, we have to decide where we want to open storage facilities, choose their sizes, and stock inventory, before any storm forms. Then after the hurricane hits in **Stage 2**, we distribute surviving supplies across a damaged transport network to meet local demand.

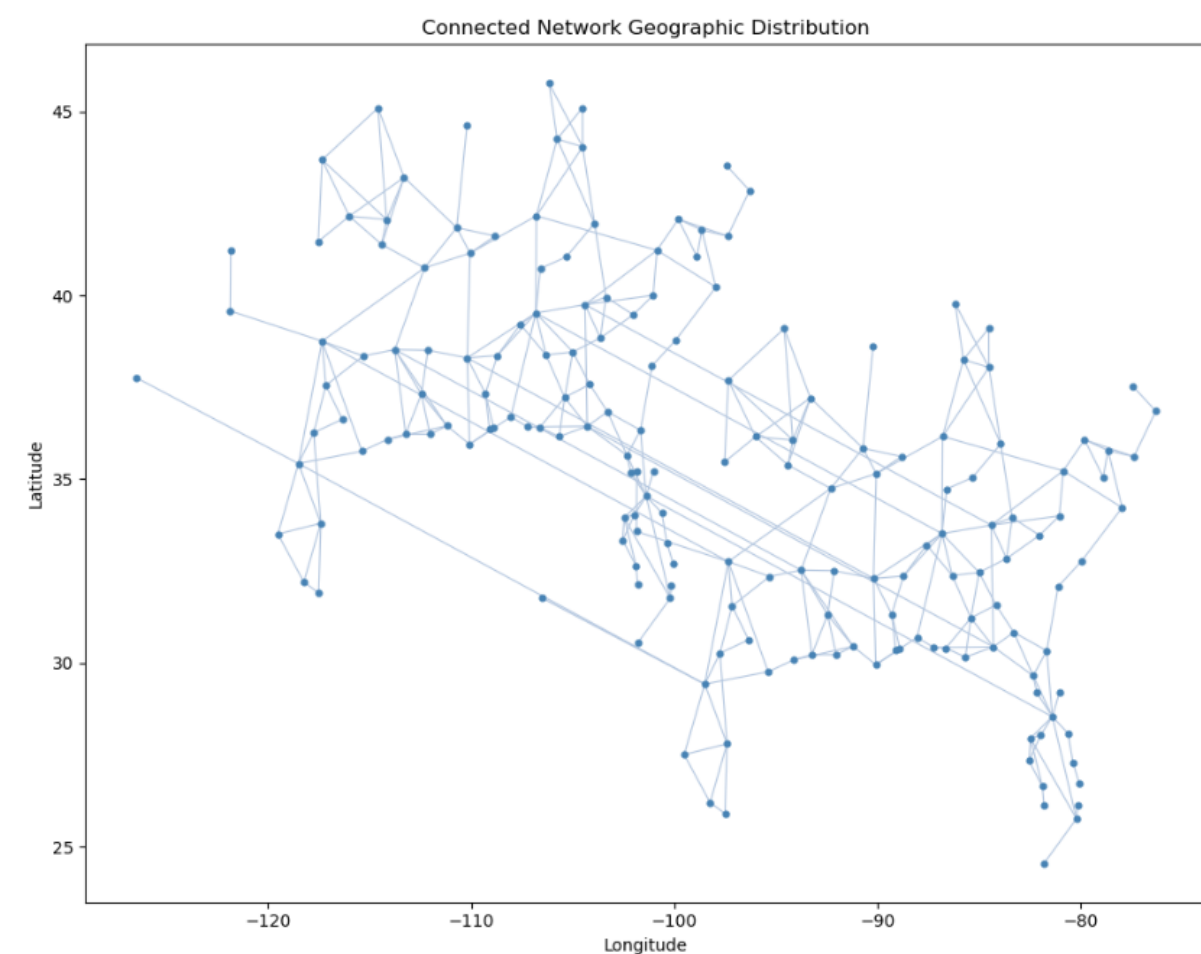


Figure 1. The 180-node transport network used in the case study, connecting candidate facility locations (nodes) via road links (edges) that carry commodity flow after a hurricane makes landfall.

The model operates over a network of 180 nodes, handling 3 distinct commodities across 3 available facility sizes. This translates to approximately 1,080 first-stage decisions. When evaluated across 500 hurricane scenarios, this results in a deterministic equivalent formulation containing roughly 1.5 million variables.

The Model

Stage 1 Decision Variables

y_{il} — binary variable to decide if we should build a facility at location i
 r_i^k — how much of commodity k to stock there

Stage 2 Decision Variables

x_{ij}^{ks} — flow of commodity k on link (i, j)
 z_i^{ks} — leftover stock
 w_i^{ks} — unmet demand

Objective Function [2]

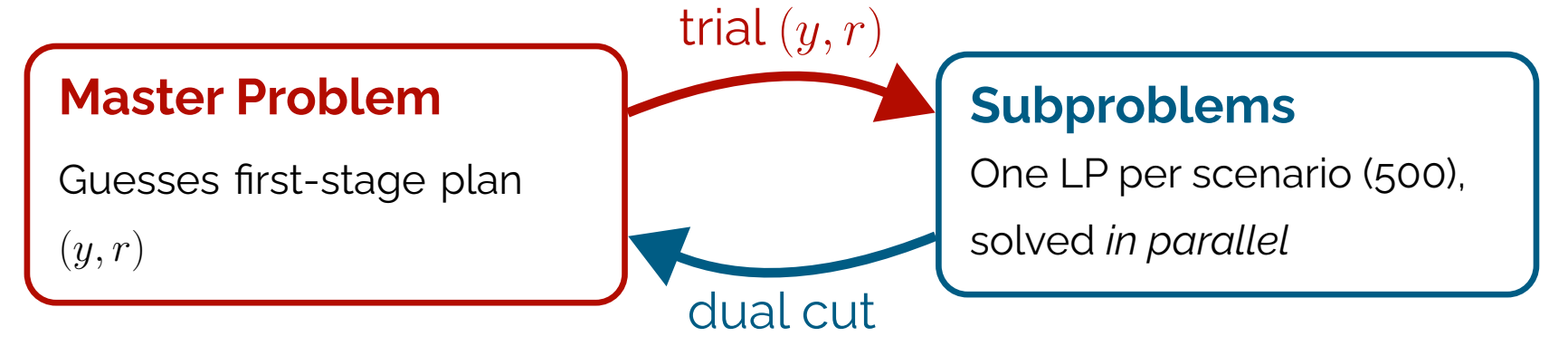
$$\min \underbrace{\sum_{i \in I} \sum_{l \in L} F_{il} y_{il} + \sum_{k \in K} \sum_{i \in I} q^k r_i^k}_{\text{first stage}} + \sum_{s \in S} P_s \left[\underbrace{\sum_{(i,j) \in A} \sum_{k \in K} c_{ij}^{ks} x_{ij}^{ks} + \sum_{i \in I} \sum_{k \in K} (h_i^k z_i^{ks} + p^k w_i^{ks})}_{\text{second stage}} \right]$$

Constraints [2]

- Supply balances at every node:** $\sum_{j \neq i} x_{ji}^{ks} + \rho_i^{ks} r_i^k - z_i^{ks} = \sum_{j \neq i} x_{ij}^{ks} + v_i^{ks} - w_i^{ks}$
- Links have limited capacity:** $\sum_{k \in K} u^k x_{ij}^{ks} \leq U_{ij}^s$
- Facilities have limited space:** $\sum_{k \in K} b^k r_i^k \leq \sum_{l \in L} M_l y_{il}$

Benders Decomposition

Benders decomposition avoids solving the whole problem at once. It splits the work into a small **master problem** handling Stage 1 only, while each scenario's Stage 2 recourse is solved separately as its own fast **subproblem** [1].



Each subproblem evaluates the guess and reports back one of two kinds of **cut**. The cuts accumulate in the master problem each iteration until we reach a maximum number of iterations or reach our set tolerance gap.

Optimality Cut: feasible subproblem → dual prices tighten the master's estimate of the true cost function from below.

Feasibility Cut: infeasible subproblem → dual rays rule out that first-stage plan, trimming the master's feasible region.

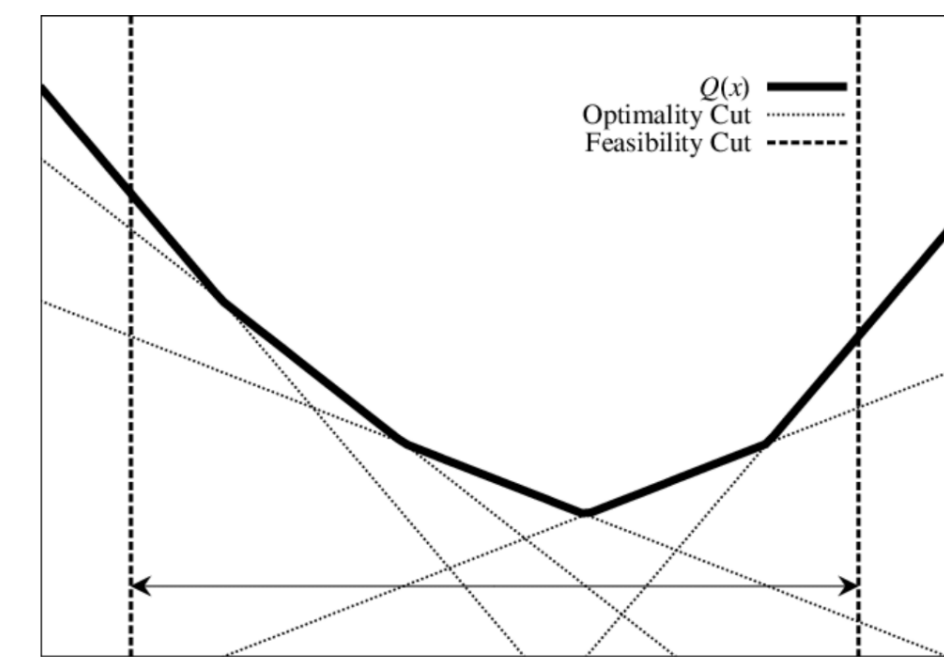
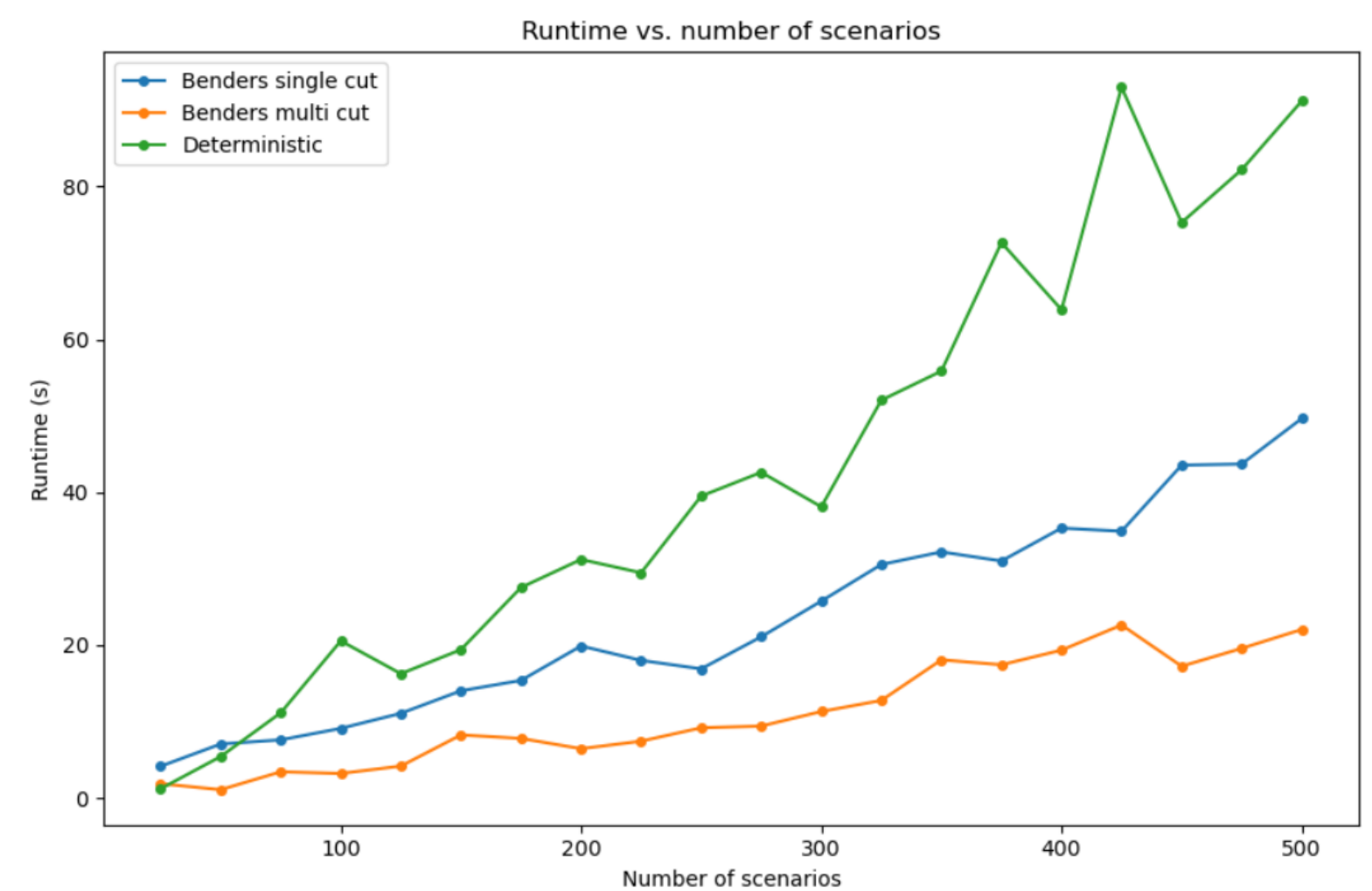


Figure 2. Illustration Of Benders Decomposition [3]

Results

The solvers are built using Gurobi. Single-cut averages all 500 scenarios into a single cut per iteration, whereas multi-cut generates one cut per scenario, adding 500 cuts each iteration.



Multi-cut Benders is easily the best performer, finishing in just 22 seconds and 7 iterations with a 1.1% gap. Adding scenario specific cuts halves the runtime compared to single-cut (49.6s) and makes it over 4× faster than the full deterministic model (91.2s), all without sacrificing solution quality.

Future Work: Three Stages

Real disasters often do not stop at one event. A hurricane can trigger a secondary disaster, so facility siting needs to plan for shocks that arrive one after another. **Stage 1** stays the same. **Stage 2** now carries its leftover stock z_i^{ks} forward instead of using it all. **Stage 3** then draws on that stock under a second scenario.

References

- [1] John R. Birge and François Louveaux. *Introduction to Stochastic Programming*. Springer, New York, 1997.
- [2] C. G. Rawls and M. A. Turnquist. Pre-positioning of emergency supplies for disaster response. *Transportation Research Part B: Methodological*, 44(4):521–534, 2010.
- [3] Takayuki Shiina, Susumu Morito, and Jun Imaizumi. Multistage stochastic programming model for optimizing allocation of running time supplements. *Journal of Advanced Mechanical Design, Systems, and Manufacturing*, 10(3):JAMDSM0043, 2016.